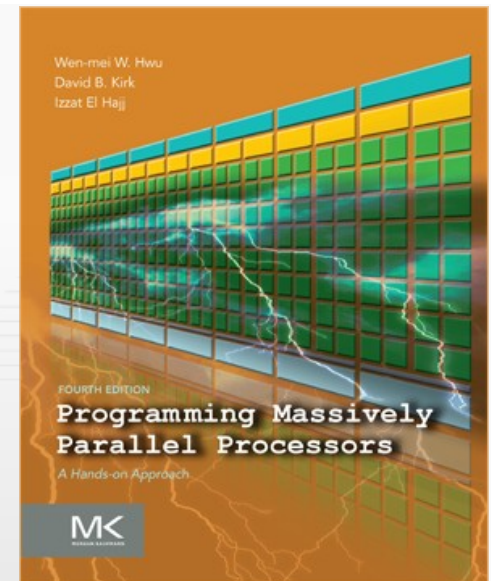


Programming Massively Parallel Processors

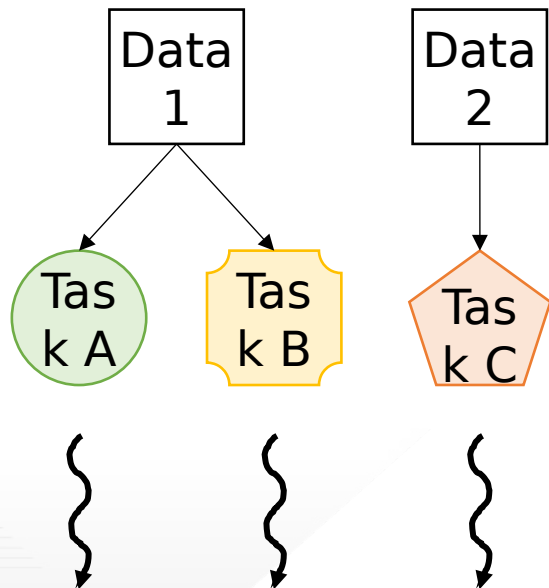
A Hands-on Approach

CHAPTER 2 > Heterogeneous Data Parallel Computing



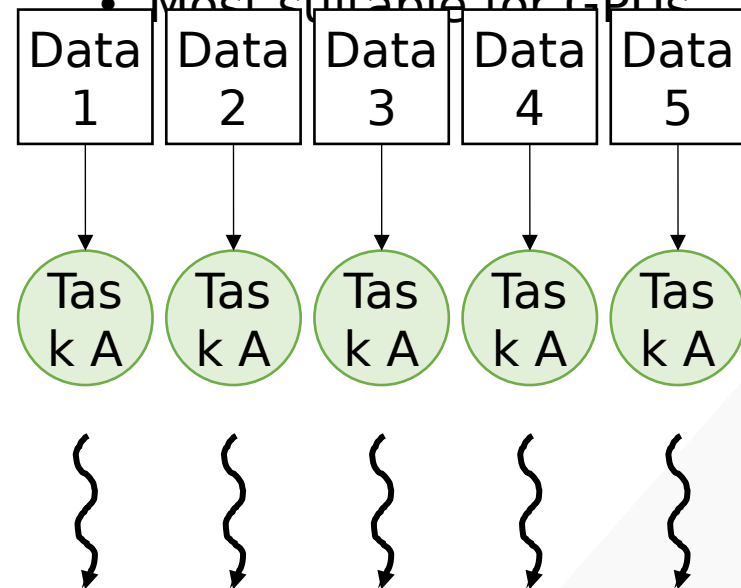
Task Parallelism

- Different operations performed on same or different data
- Usually a modest number of tasks unleashing a modest amount of parallelism

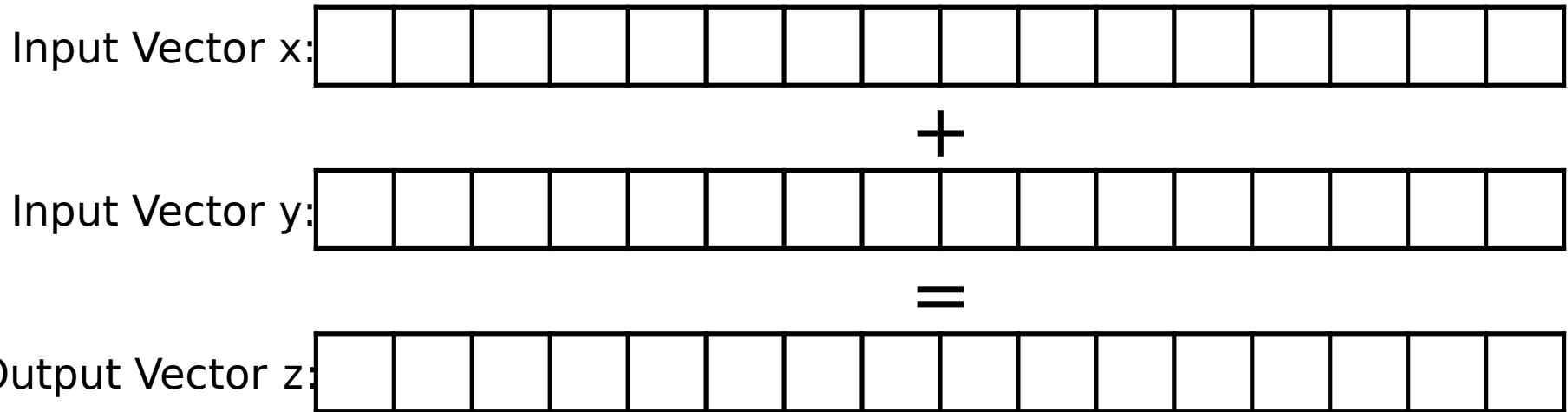


Data Parallelism

- Same operations performed on different data
- Potentially massive amounts of data unleashing massive amounts of parallelism
- Most suitable for GPUs

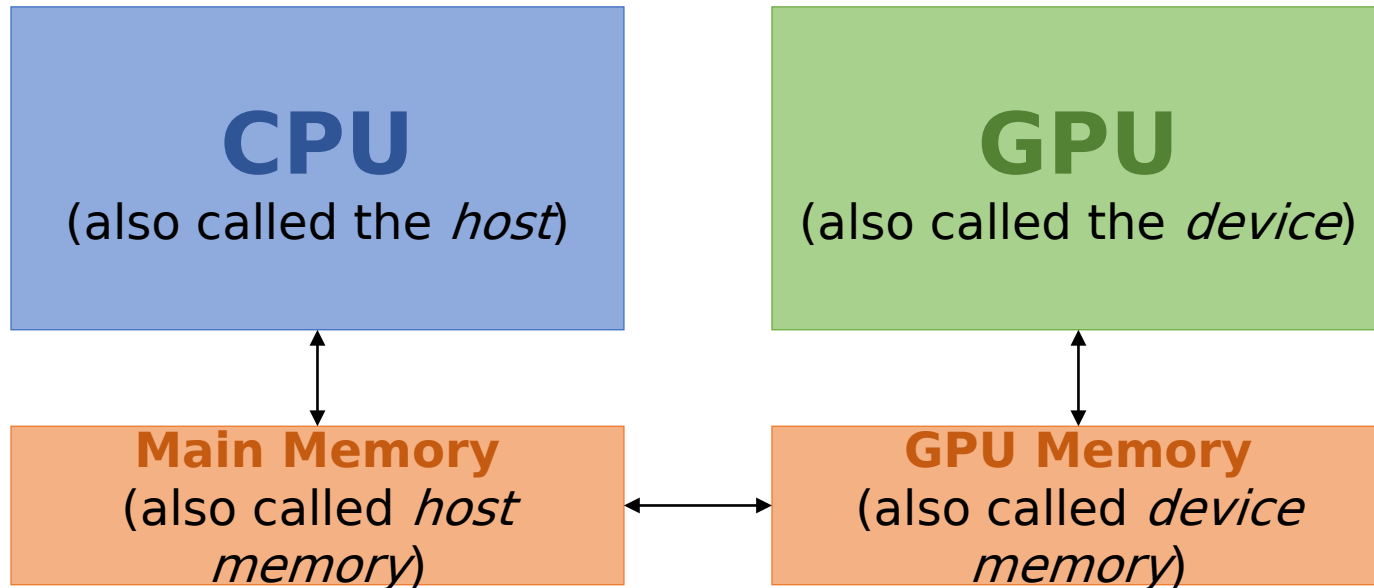


- Vector addition
 - The “Hello, World!” for data parallel programming



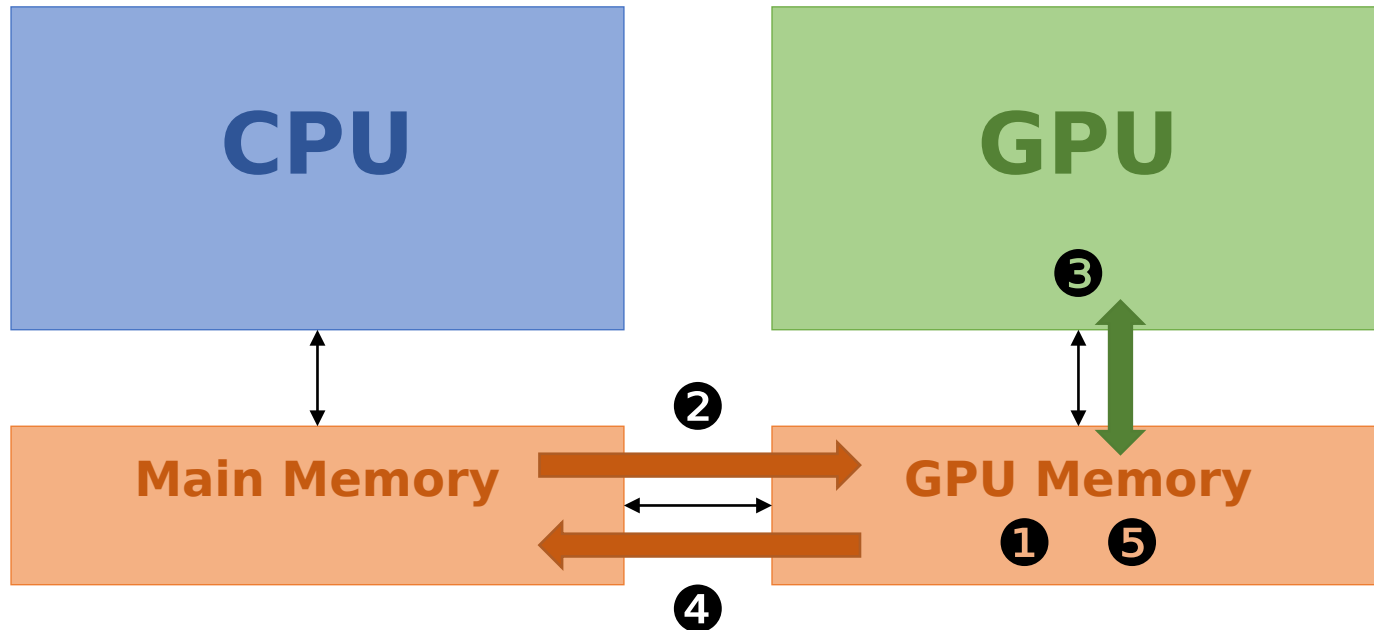
```
void vecadd(float* x, float* y, float* z, int N) {  
    for(unsigned int i = 0; i < N; ++i) {  
        z[i] = x[i] + y[i];  
    }  
}
```

Sequential C Code



The CPU and GPU have separate memories can cannot access each others' memories

Need to transfer data between them



- ① Allocate GPU memory
- ② Copy data to GPU memory
- ③ Perform computation on GPU
- ④ Copy data from GPU memory
- ⑤ Deallocate GPU memory

- Allocating memory:

```
cudaError_t cudaMalloc(void **devPtr, size_t size)
```

- devPtr: Pointer to pointer to allocated device memory
- size: Requested allocation size in bytes

- Deallocating memory:

```
cudaError_t cudaFree(void *devPtr)
```

- devPtr: Pointer to device memory to free

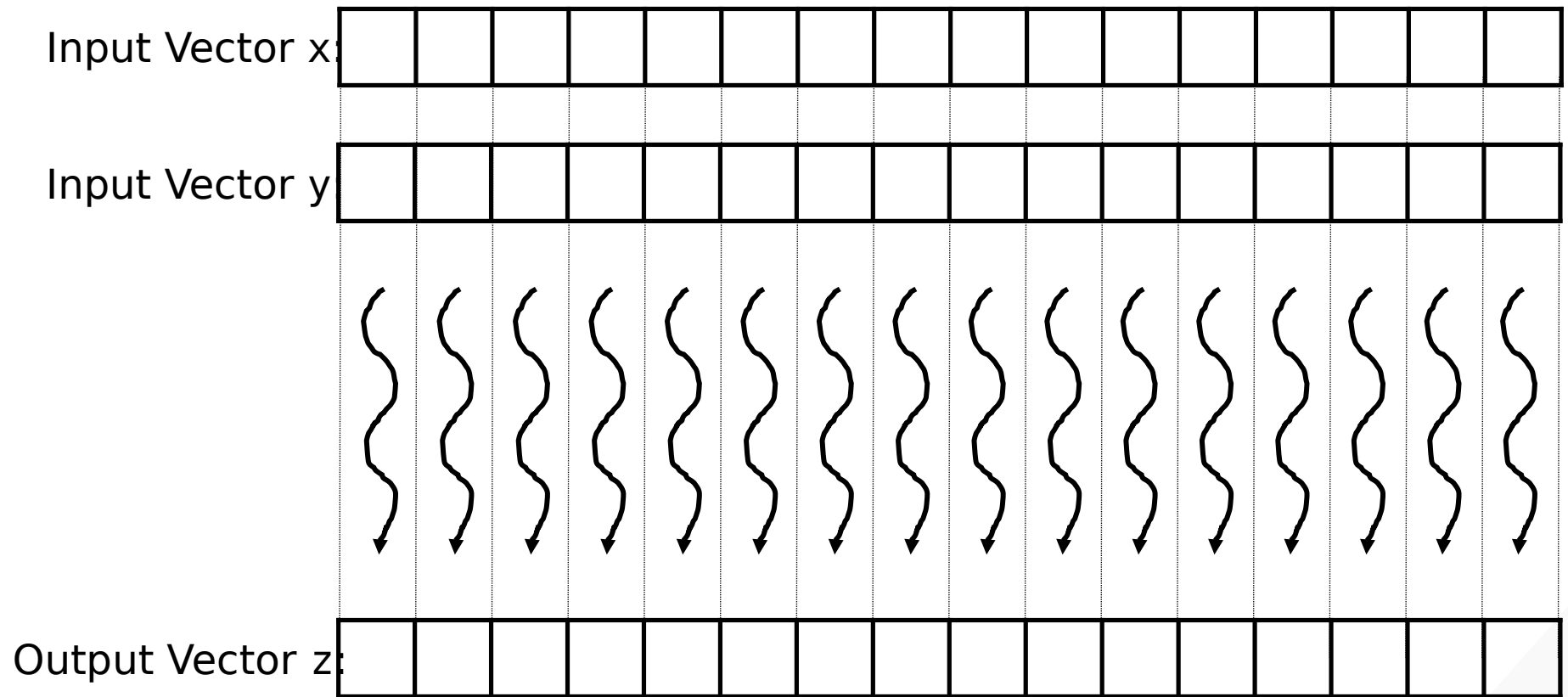
- Return type: cudaError_t
 - Helps with error checking (discussed later)

- Copying memory:

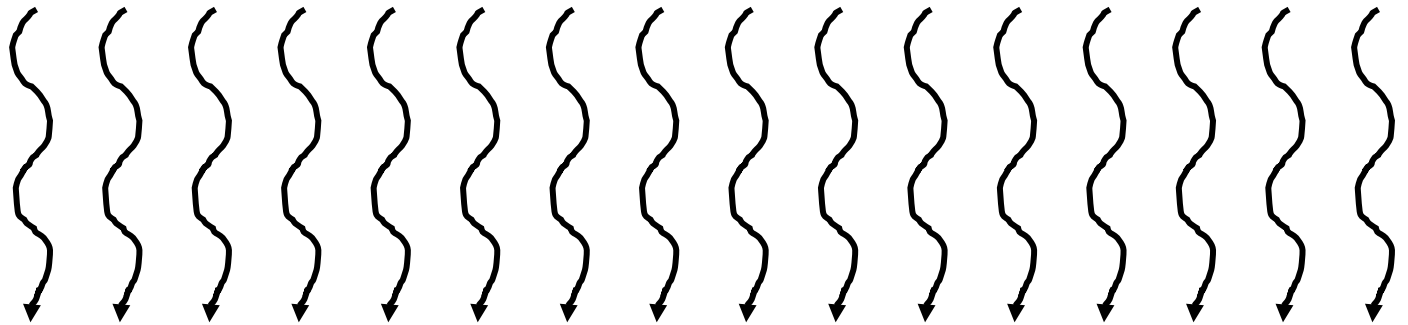
```
cudaError_t cudaMemcpy(void *dst, const void *src,  
                        size_t count, enum cudaMemcpyKind kind)
```

- dst: Destination memory address
- src: Source memory address
- count: Size in bytes to copy
- kind: Type of transfer
 - cudaMemcpyHostToHost
 - cudaMemcpyHostToDevice
 - cudaMemcpyDeviceToHost
 - cudaMemcpyDeviceToDevice

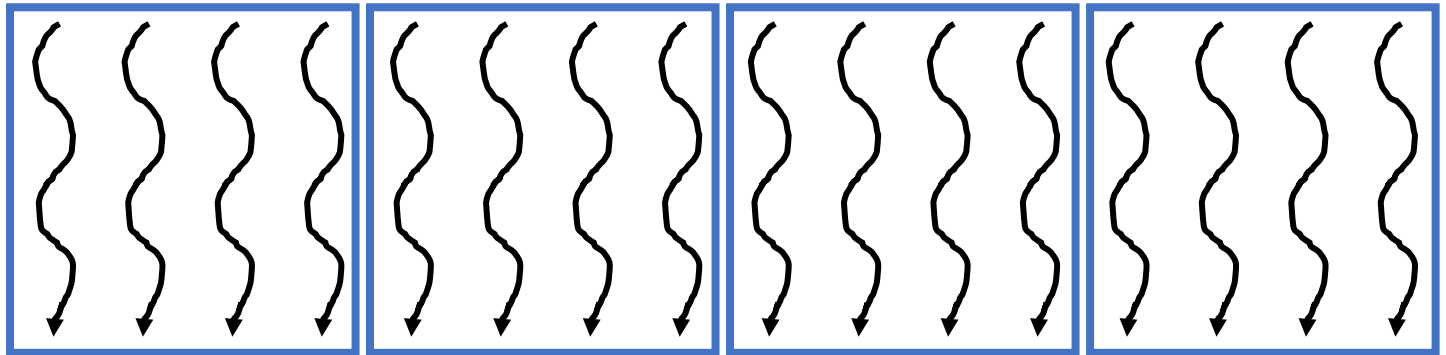
```
void vecadd(float* x, float* y, float* z, int N) {  
  
    // Allocate GPU memory  
    float *x_d, *y_d, *z_d;  
    cudaMalloc((void**) &x_d, N*sizeof(float));  
    cudaMalloc((void**) &y_d, N*sizeof(float));  
    cudaMalloc((void**) &z_d, N*sizeof(float));  
  
    // Copy data to GPU memory  
    cudaMemcpy(x_d, x, N*sizeof(float),  
cudaMemcpyHostToDevice);  
    cudaMemcpy(y_d, y, N*sizeof(float),  
cudaMemcpyHostToDevice);  
  
    // Perform computation on GPU  
    ...  
  
    // Copy data from GPU memory  
    cudaMemcpy(z, z_d, N*sizeof(float),  
cudaMemcpyDeviceToHost);  
  
    // Deallocate GPU memory  
    cudaFree(x_d);  
    cudaFree(y_d);  
    cudaFree(z_d);  
}
```

Simple strategy: assign one GPU **thread** per vector element



An array of GPU threads is called a **grid**



Threads in a grid are grouped into
thread **blocks**

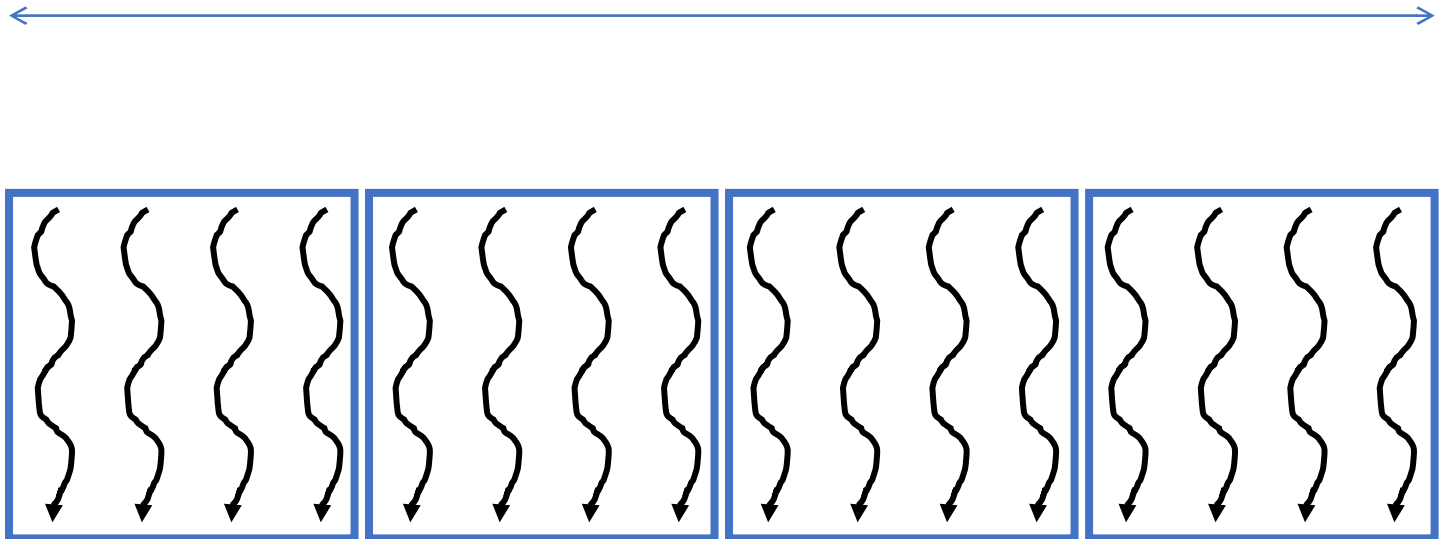
(significance: threads in the same block can
collaborate in ways that threads in different blocks
cannot – discussed later)

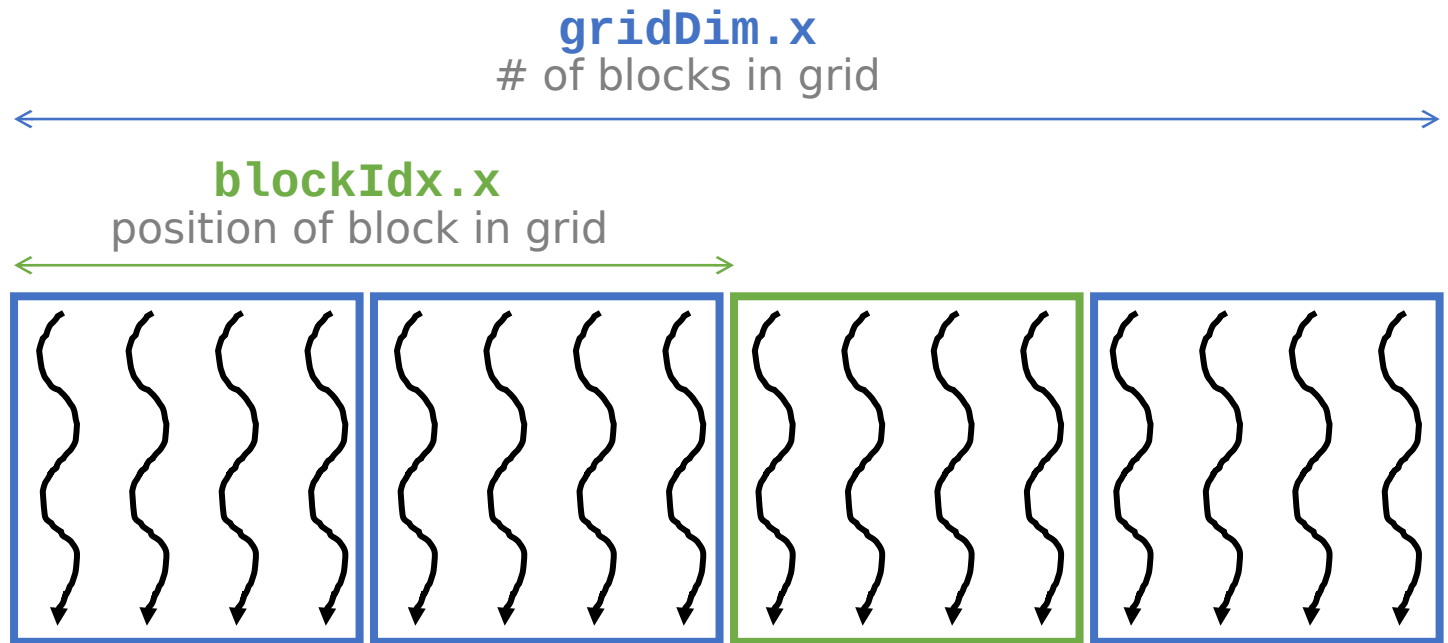
- Threads in the same grid execute the same function known as a **kernel**
- A grid can be launched by calling a kernel and configuring it with appropriate grid and block sizes
- Example:

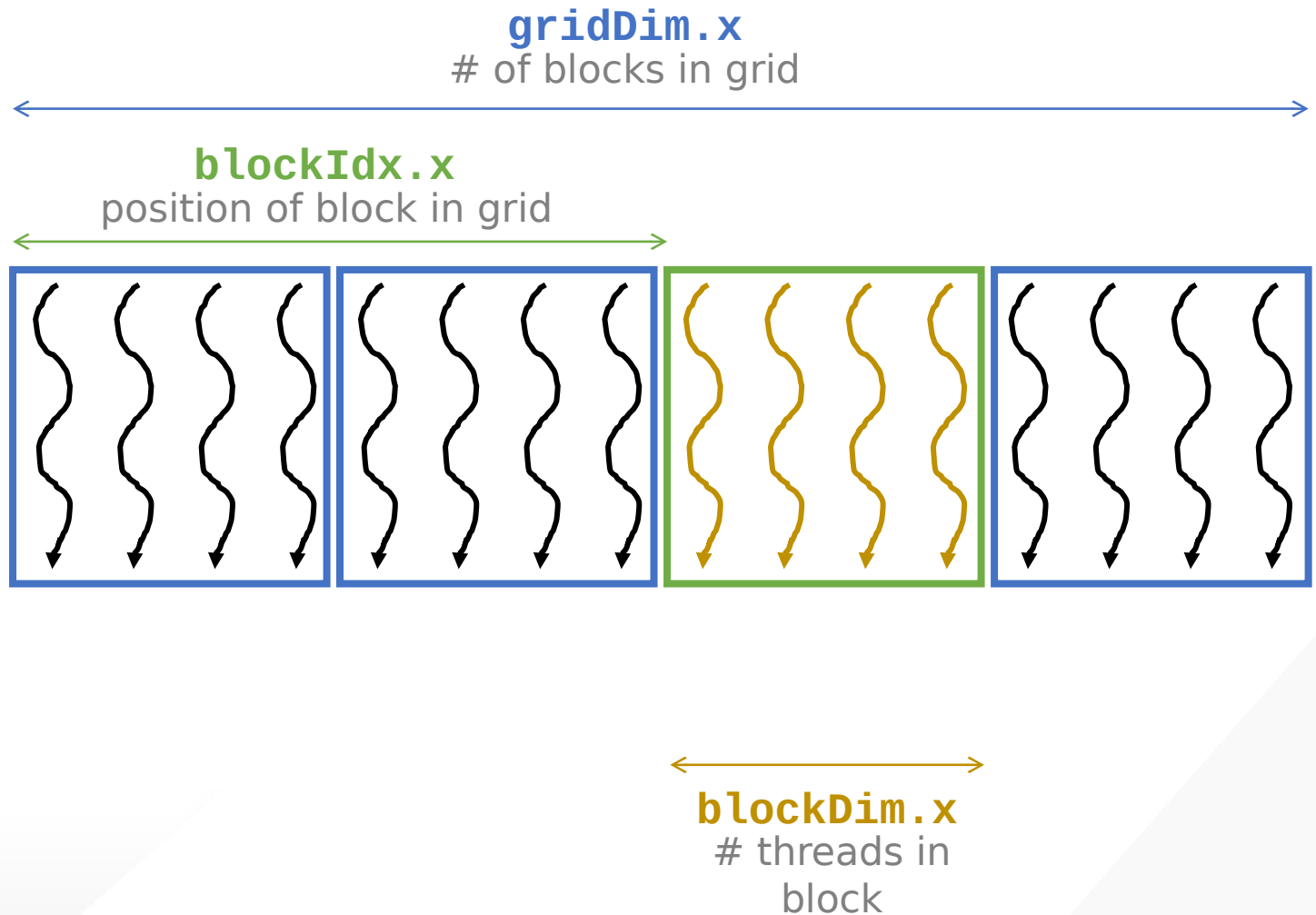
```
const unsigned int numThreadsPerBlock = 512;  
const unsigned int numBlocks = N/numThreadsPerBlock;  
vecadd_kernel <<< numBlocks, numThreadsPerBlock >>> (x_d, y_d, z_d,  
N);
```

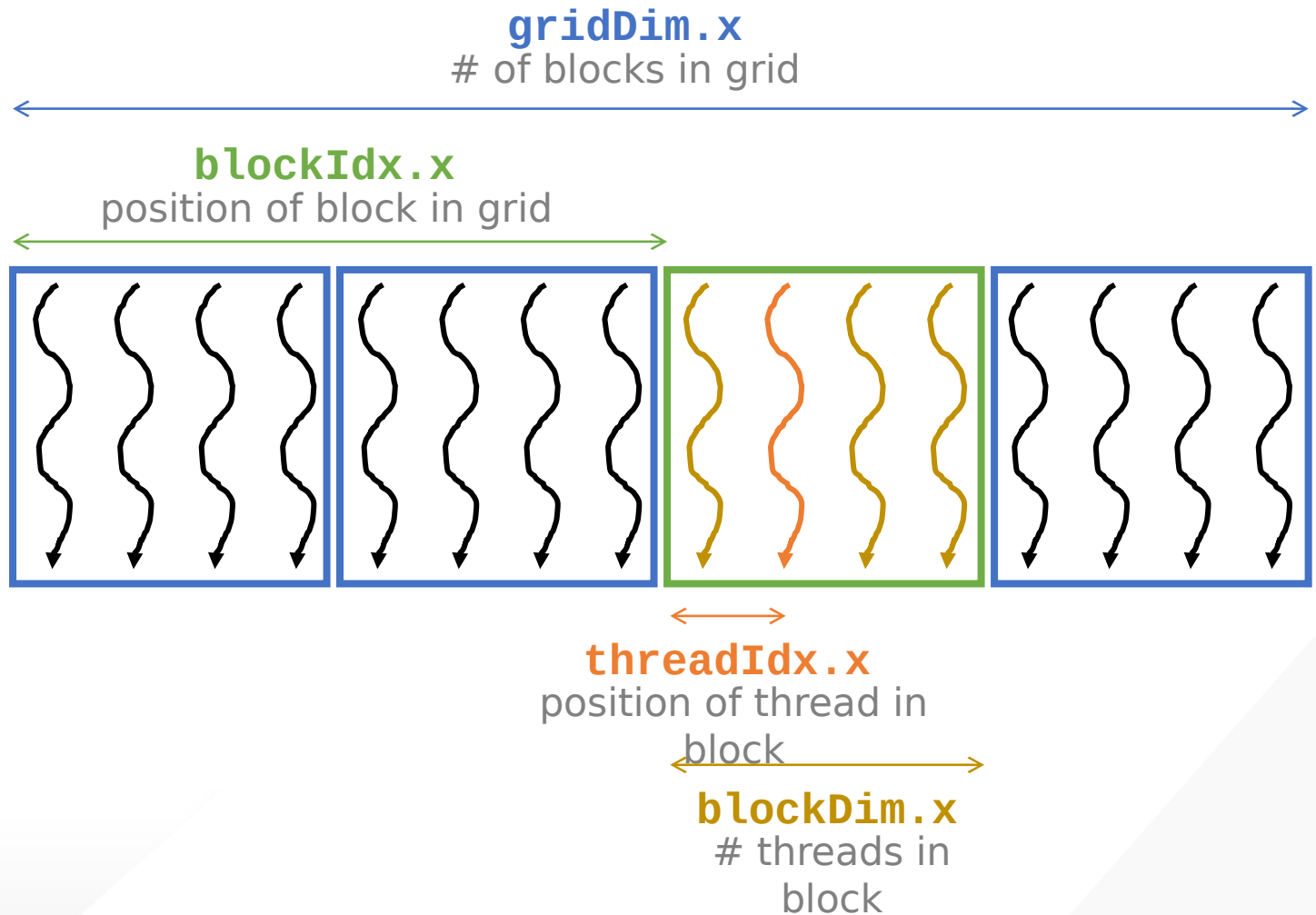
- A kernel is similar to a C/C++ function
- It is preceded by the keyword `__global__` to indicate that it is a GPU kernel
- It uses special keywords to distinguish different threads from each other

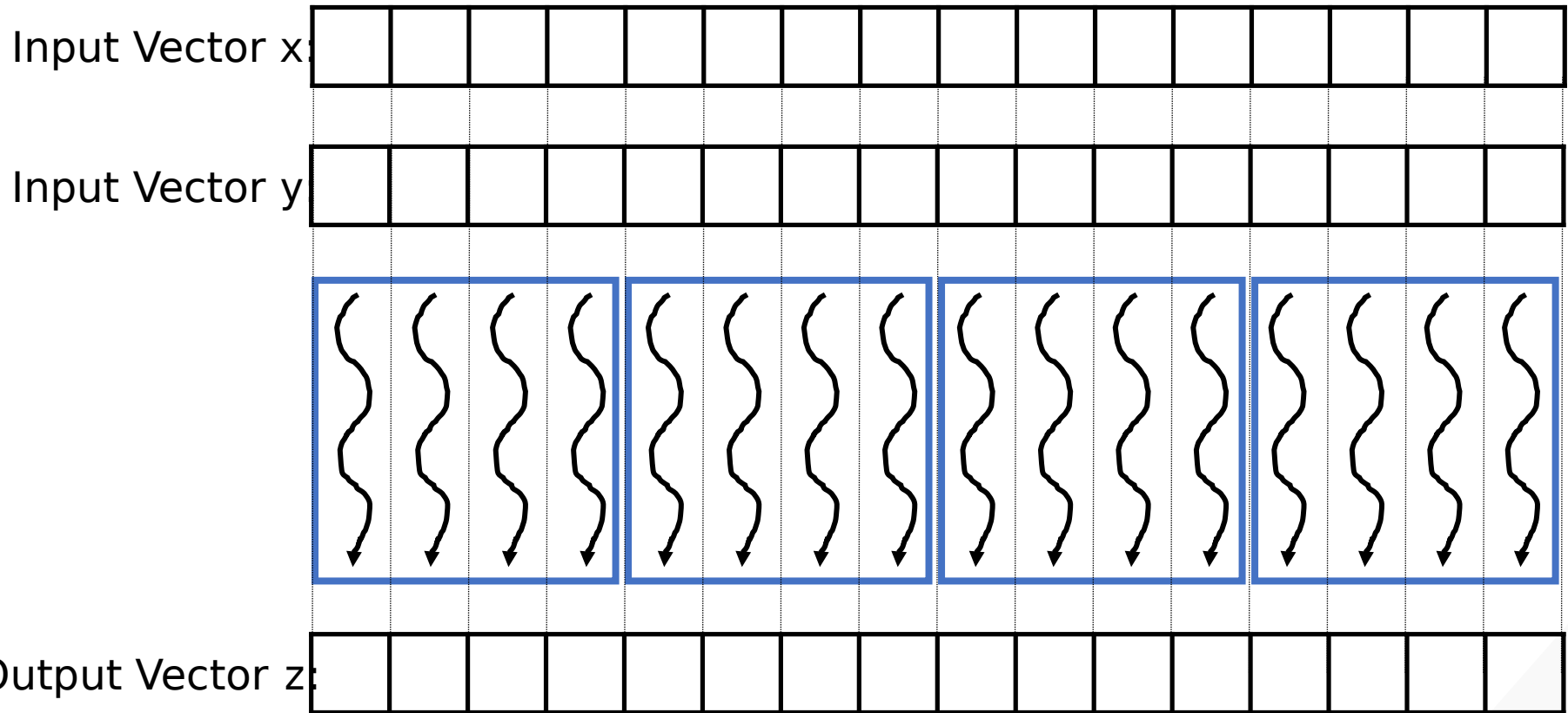
gridDim.x
of blocks in grid











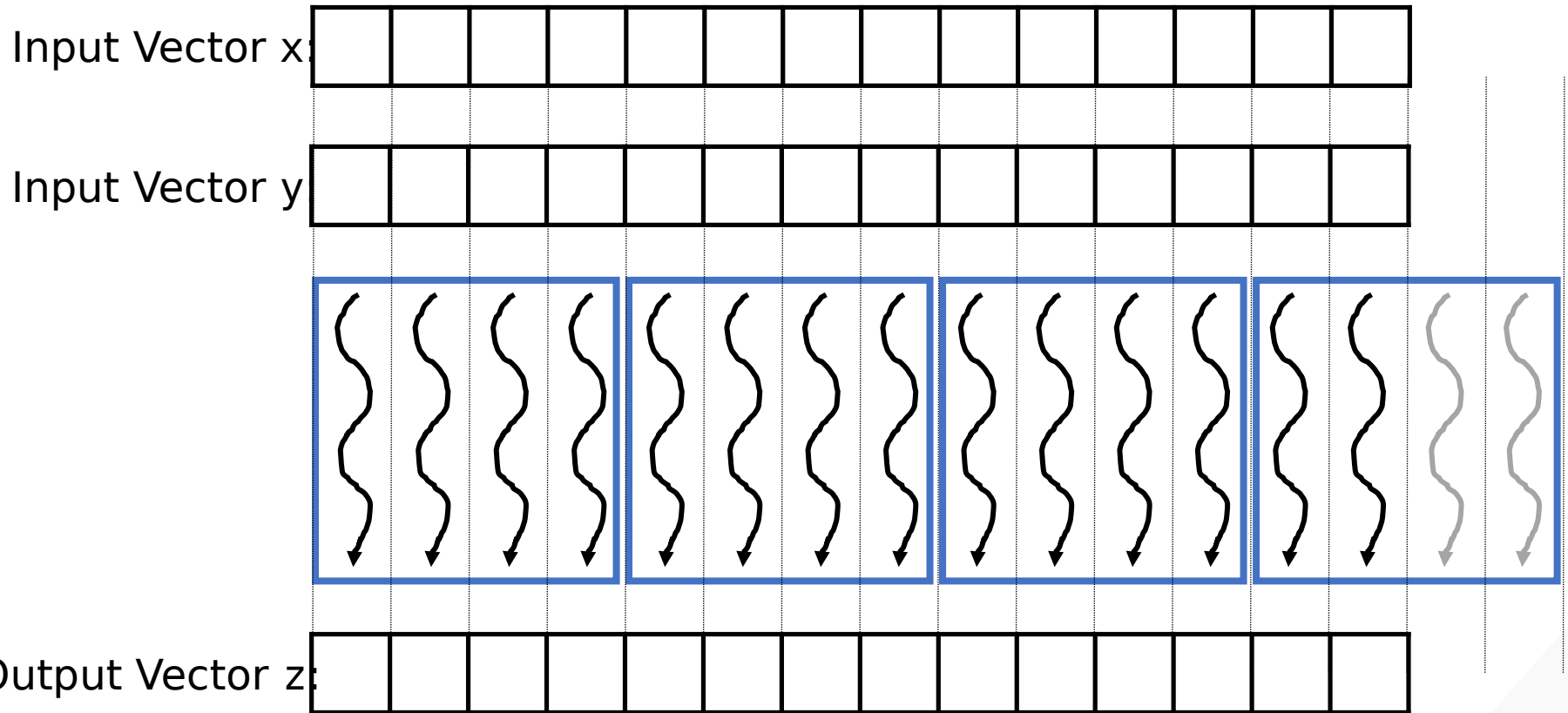
```
__global__ void vecadd_kernel(float* x, float* y, float* z, int N) {
    int i = blockDim.x*blockIdx.x + threadIdx.x;
    z[i] = x[i] + y[i];
}
```

Identify a thread's global index and use it to index the arrays

- Recall launch configurations:

```
const unsigned int numThreadsPerBlock = 512;  
const unsigned int numBlocks = N/numThreadsPerBlock;  
vecadd_kernel <<< numBlocks, numThreadsPerBlock >>> (x_d, y_d, z_d,  
N);
```

- If N is not a multiple of numThreadsPerBlock, fewer threads will be launched than desired
- Solution: use the ceiling to launch extra threads then omit the threads after the boundary

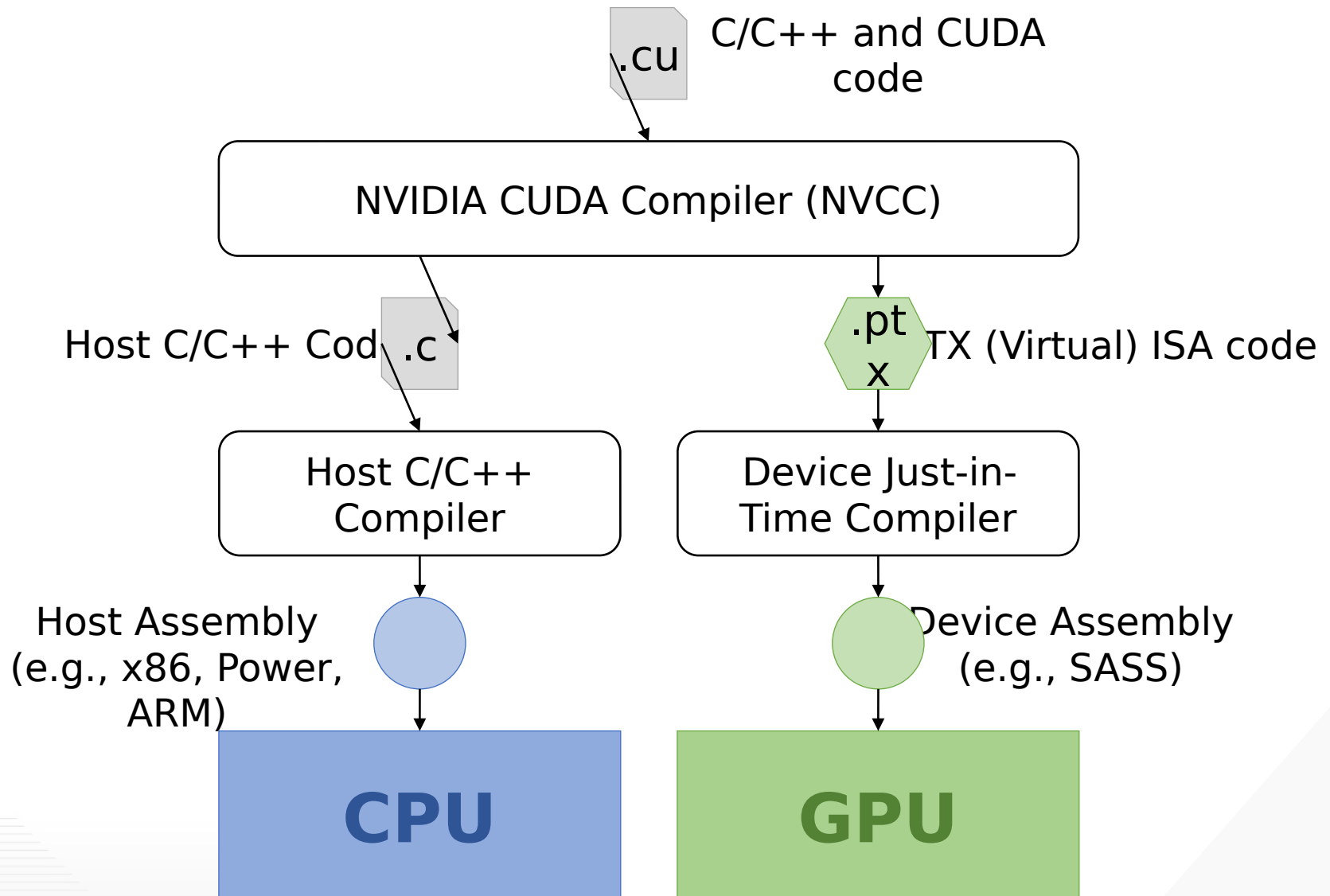


- Launch code:

```
const unsigned int numBlocks = (N + numThreadsPerBlock - 1)/numThreadsPerBlock;
```

- Kernel code

```
__global__ void vecadd_kernel(float* x, float* y, float* z, int N) {  
    int i = blockDim.x*blockIdx.x + threadIdx.x;  
    if(i < N) {  
        z[i] = x[i] + y[i];  
    }  
}
```



- Keywords that differentiate where a function is intended to run

Keyword	Callable From	Executed On
<code>__host__</code> (default)	Host	Host
<code>__global__</code>	Host (or Device)	Device
<code>__device__</code>	Device	Device

- Why need `__host__` if it is the default?

- The keyword `__host__` is useful when needing to mark a function as executable on both the host and the device

```
__host__ __device__ float f(float a, float b) {  
    return a + b;  
}  
  
void vecadd(float* x, float* y, float* z, int N) {  
    for(unsigned int i = 0; i < N; ++i) {  
        z[i] = f(x[i], y[i]);  
    }  
}  
  
__global__ void vecadd_kernel(float* x, float* y, float* z, int N) {  
    int i = blockDim.x*blockIdx.x + threadIdx.x;  
    if (i < N) {  
        z[i] = f(x[i], y[i]);  
    }  
}
```


How much faster?

- By default, kernel calls are asynchronous
 - Useful for overlapping GPU computations with CPU computations
- Use the following API function to synchronize
`cudaError_t cudaDeviceSynchronize()`
 - Blocks until the device has completed all preceding requested tasks

- All CUDA API calls return an error code `cudaError_t` that can be used to check if any errors occurred
 - Example:

```
cudaError_t err = ...;
if(err != cudaSuccess) {
    printf("Error: %s\n", cudaGetErrorString(err));
    exit(0);
}
```

- For kernel calls, one can check the error returned by `cudaDeviceSynchronize()` or call the following API function:
`cudaError_t cudaGetLastError()`

- The rest of this course is about parallelization strategies and optimizations for massively parallel processors

- Wen-mei W. Hwu, David B. Kirk, and Izzat El Hajj. *Programming Massively Parallel Processors: A Hands-on Approach*. Morgan Kaufmann, 2022.
- CUDA C++ Programming Guide:
<https://docs.nvidia.com/cuda/cuda-c-programming-guide>